

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import fft2, fftshift

# Simulation Parameters
size = 100 # Grid size
steps = 5000 # Evolution steps
diffusion = 0.1 # Fixed diffusion rate
perturbation_levels = [0.01, 0.1, 0.5] # Increasing perturbation levels

# Function to create different initial geometries
def create_geometry(shape, size):
    grid = np.zeros((size, size))

    if shape == "nested_rings":
        for r in range(10, size//2, 20):
            y, x = np.ogrid[-size//2:size//2, -size//2:size//2]
            mask = (x**2 + y**2 >= r**2) & (x**2 + y**2 < (r+4)**2)
            grid[mask] = 1

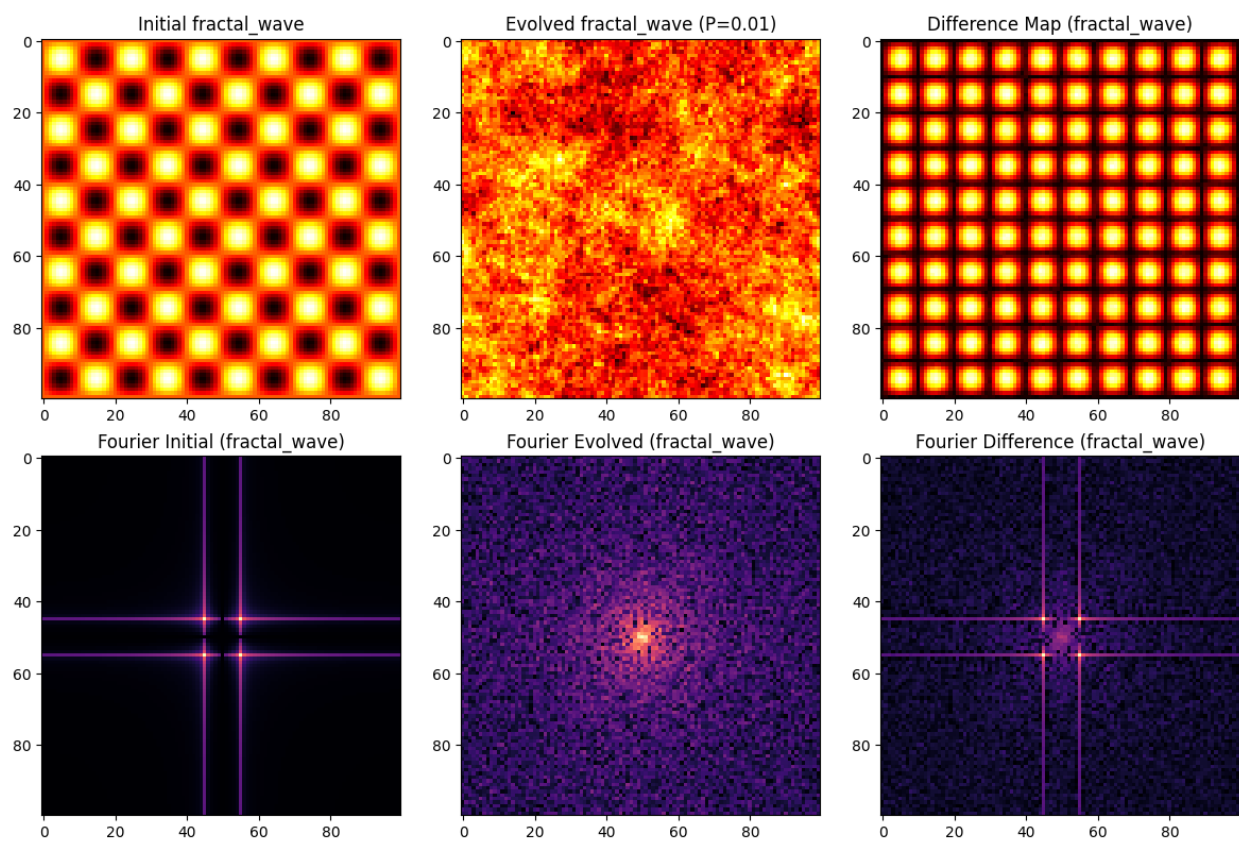
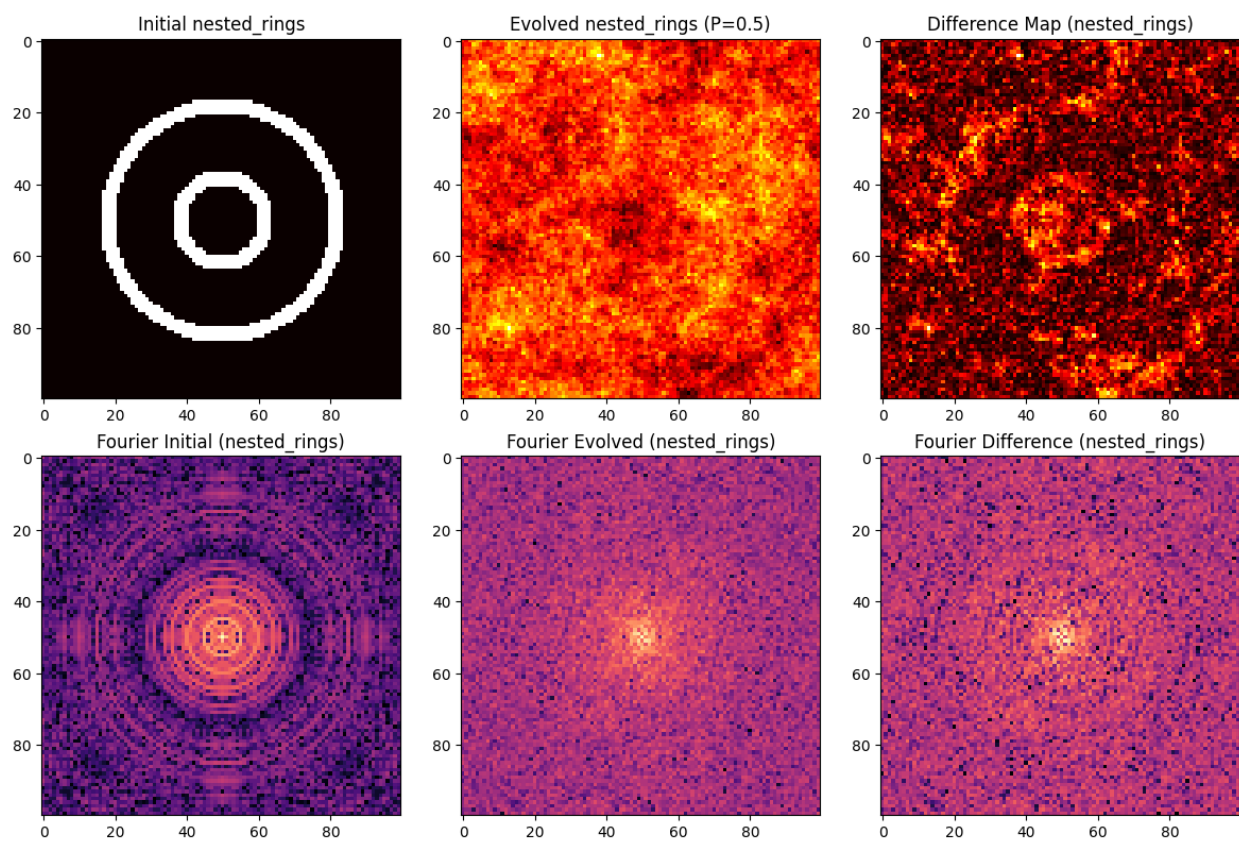
    elif shape == "fractal_wave":
        x = np.linspace(0, 2*np.pi, size)
        y = np.linspace(0, 2*np.pi, size)
        X, Y = np.meshgrid(x, y)
        grid = np.sin(5 * X) * np.sin(5 * Y) # Fractal pattern

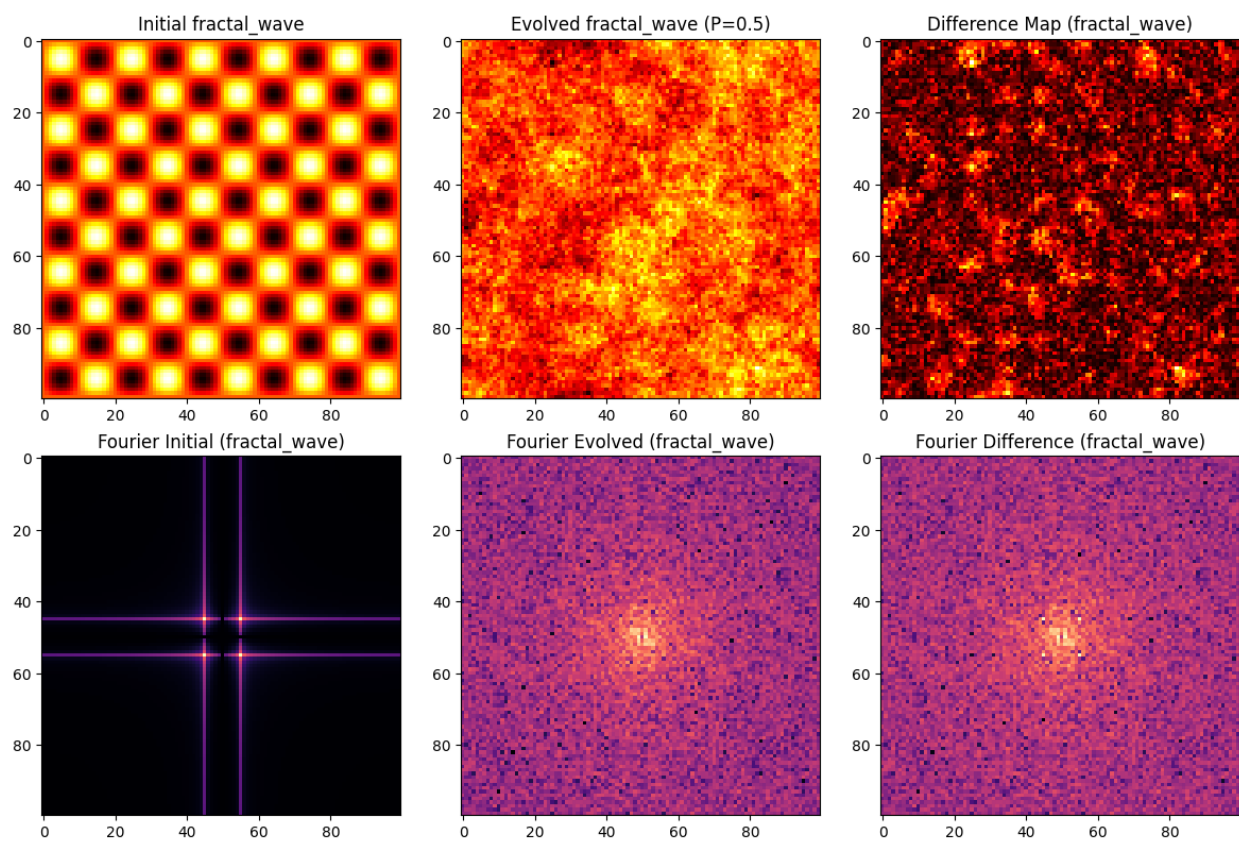
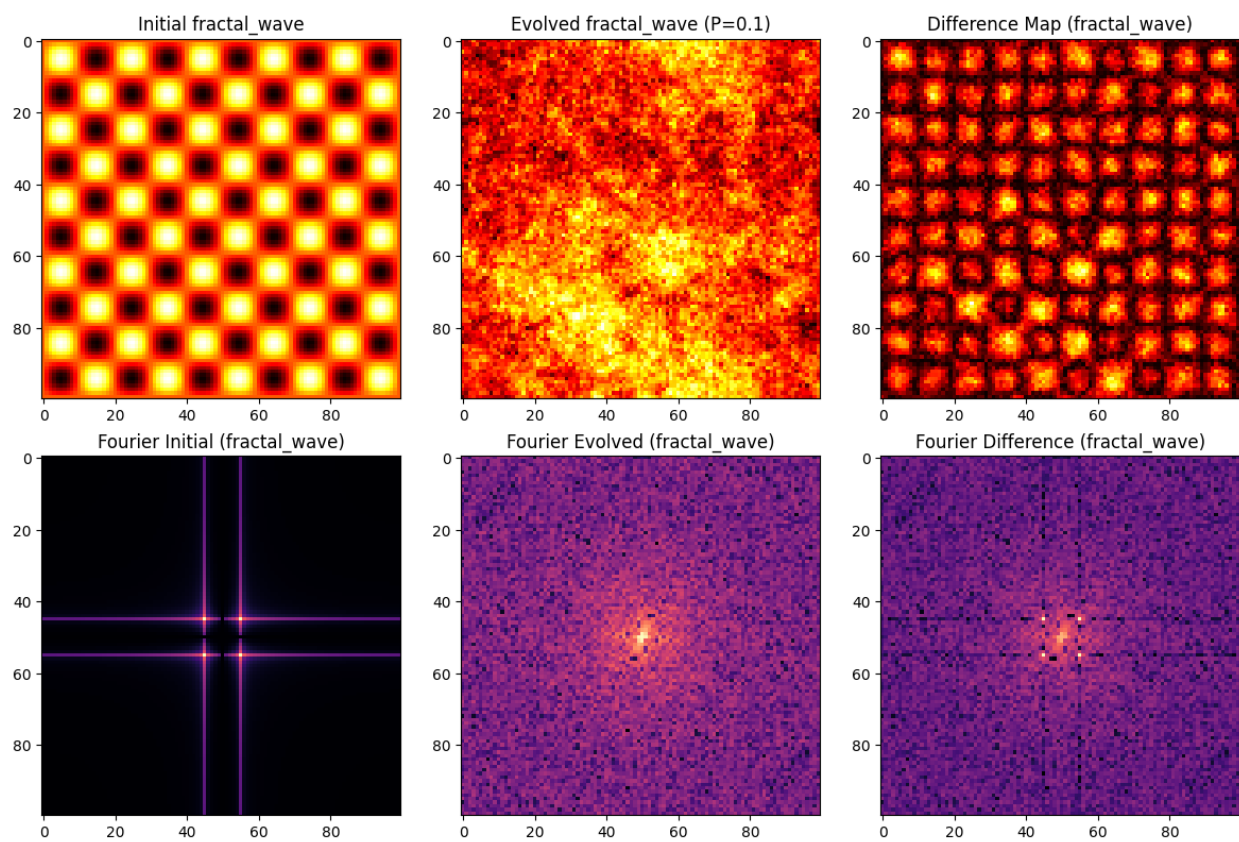
    elif shape == "toroidal_fractal":
        fractal = create_geometry("fractal_wave", size)
        rings = create_geometry("nested_rings", size)
        grid = fractal * rings # Combine toroidal & fractal structures

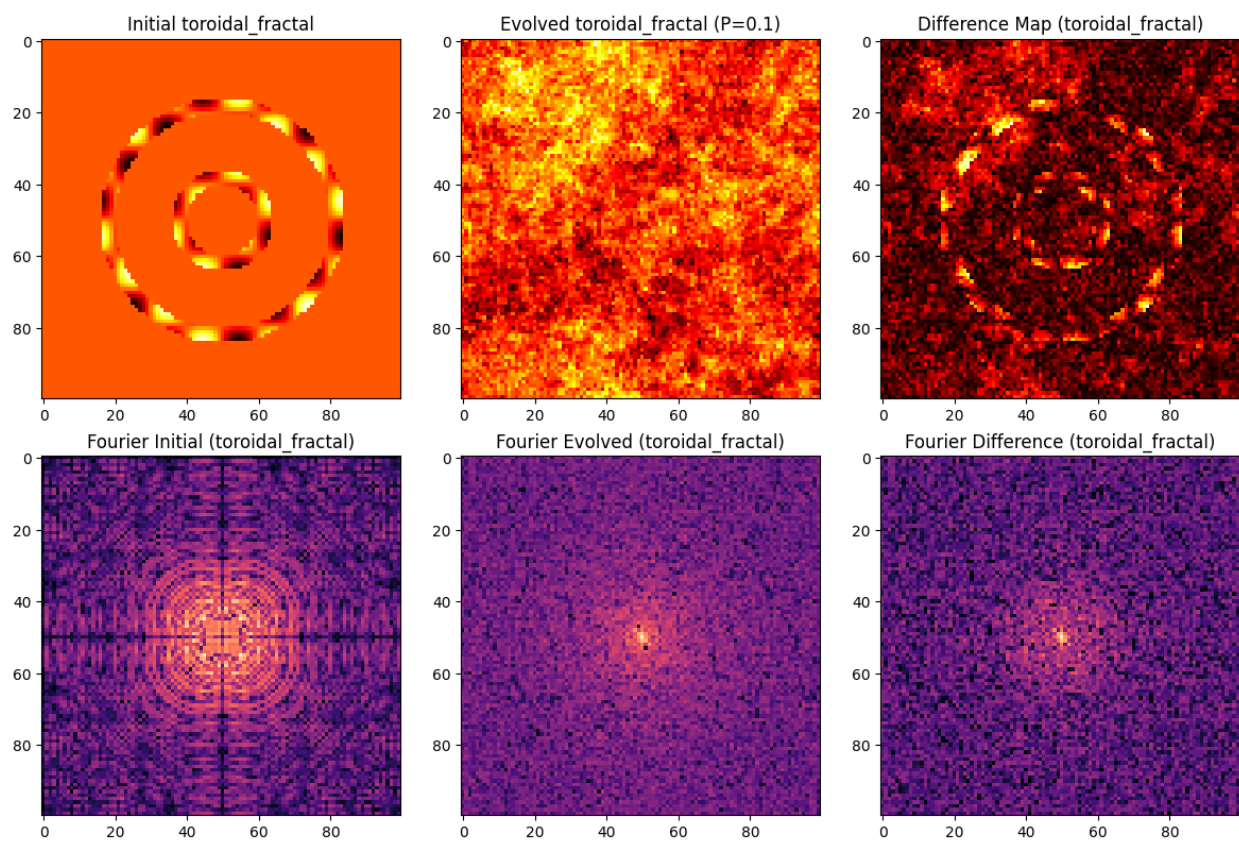
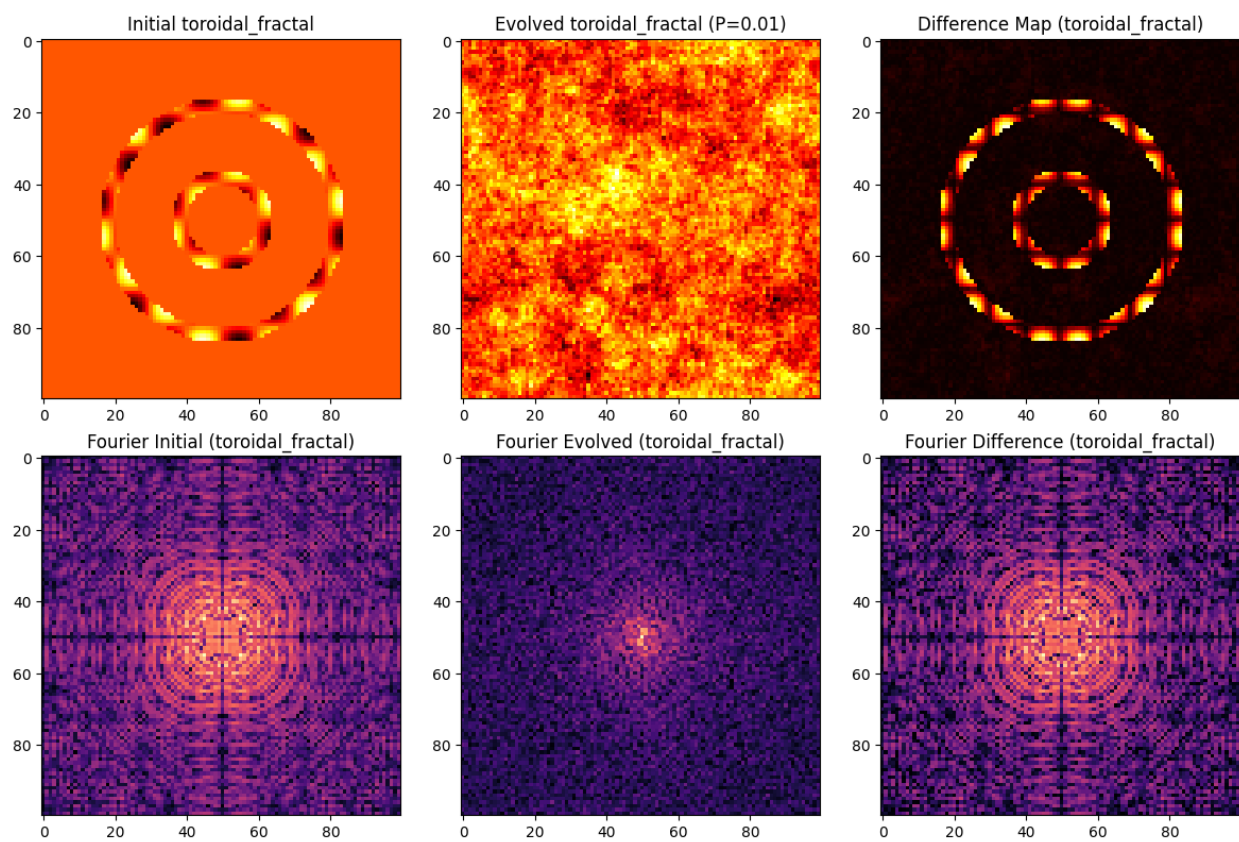
    return grid

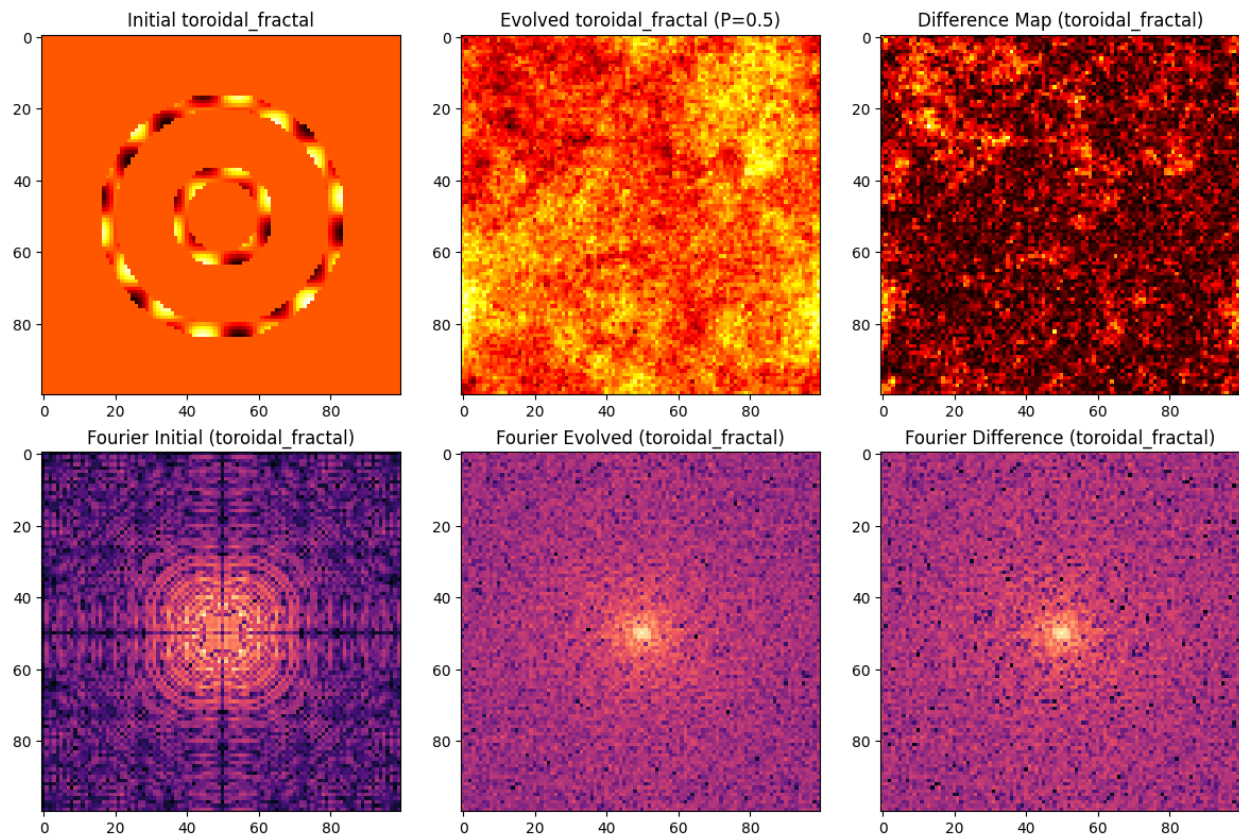
# Evolution function with perturbations
def evolve(grid, diffusion, perturbation, steps):
    evolved = grid.copy()
    for _ in range(steps):
        laplacian = (
            np.roll(evolved, 1, axis=0) + np.roll(evolved, -1, axis=0) +

```









```

    np.roll(evolved, 1, axis=1) + np.roll(evolved, -1, axis=1) -
    4 * evolved
)
    evolved += diffusion * laplacian + perturbation * np.random.randn(*evolved.shape)
return evolved

```

Visualization function

```
def plot_results(initial, evolved, shape, perturbation):
```

```
    fig, axes = plt.subplots(2, 3, figsize=(12, 8))
```

Initial condition

```
axes[0, 0].imshow(initial, cmap="hot")
```

```
axes[0, 0].set_title(f"Initial {shape}")
```

Evolved state

```
axes[0, 1].imshow(evolved, cmap="hot")
```

```
axes[0, 1].set_title(f"Evolved {shape} (P={perturbation})")
```

Difference map

```
difference = np.abs(initial - evolved)
```



```

axes[0, 2].imshow(difference, cmap="hot")
axes[0, 2].set_title(f"Difference Map ({shape})")

# Fourier transforms
ft_initial = fftshift(np.abs(fft2(initial)))
ft_evolved = fftshift(np.abs(fft2(evolved)))

axes[1, 0].imshow(np.log(1 + ft_initial), cmap="magma")
axes[1, 0].set_title(f"Fourier Initial ({shape})")

axes[1, 1].imshow(np.log(1 + ft_evolved), cmap="magma")
axes[1, 1].set_title(f"Fourier Evolved ({shape})")

ft_diff = np.abs(ft_initial - ft_evolved)
axes[1, 2].imshow(np.log(1 + ft_diff), cmap="magma")
axes[1, 2].set_title(f"Fourier Difference ({shape})")

plt.tight_layout()
plt.show()

# Run experiments for all geometries and perturbations
geometries = ["nested_rings", "fractal_wave", "toroidal_fractal"]

for shape in geometries:
    for perturbation in perturbation_levels:
        initial = create_geometry(shape, size)
        evolved = evolve(initial, diffusion, perturbation, steps)
        plot_results(initial, evolved, shape, perturbation)

```

